

SYSTEM MANUAL

VISITORPASS PORTAL

Visitor Management & Letter Tracking System

Complete architecture reference, API schemas, database designs, and administrator guidelines for the Punit Joshi Visitor Management Portal.

CLIENT / CONTEXT

Punit Joshi Office

FRAMEWORK & DB

Next.js 14 & MongoDB

GENERATED DATE

July 1, 2026

1. Introduction & Overview

The **Visitor Management & Letter Tracking System** is a full-stack Next.js application built specifically for the administrative office of **Punit Joshi**. The application acts as a digital gatekeeper and workflow optimizer, replacing manual registries with an elegant, responsive, and secure platform.

The core objectives of the system are:

- **Visitor Registration:** Seamless onboarding for guests through a multi-step registration form supporting custom fields depending on the purpose of the visit (medical, education, employment, utility issues, etc.).
- **Worker Tracking:** Maintain records of party workers, their positions, booth distributions, and detailed demographics for coordination.
- **Letter Tracking:** Keep record of all inward and outward letters, assigning responsibilities and tracking completion statuses.
- **Scheduling & Events:** Integrated event booking request system and public calendar tracking for administrative tasks.
- **Granular Role Security:** Permission-based navigation rendering separate layouts and actions for administrators and general users.

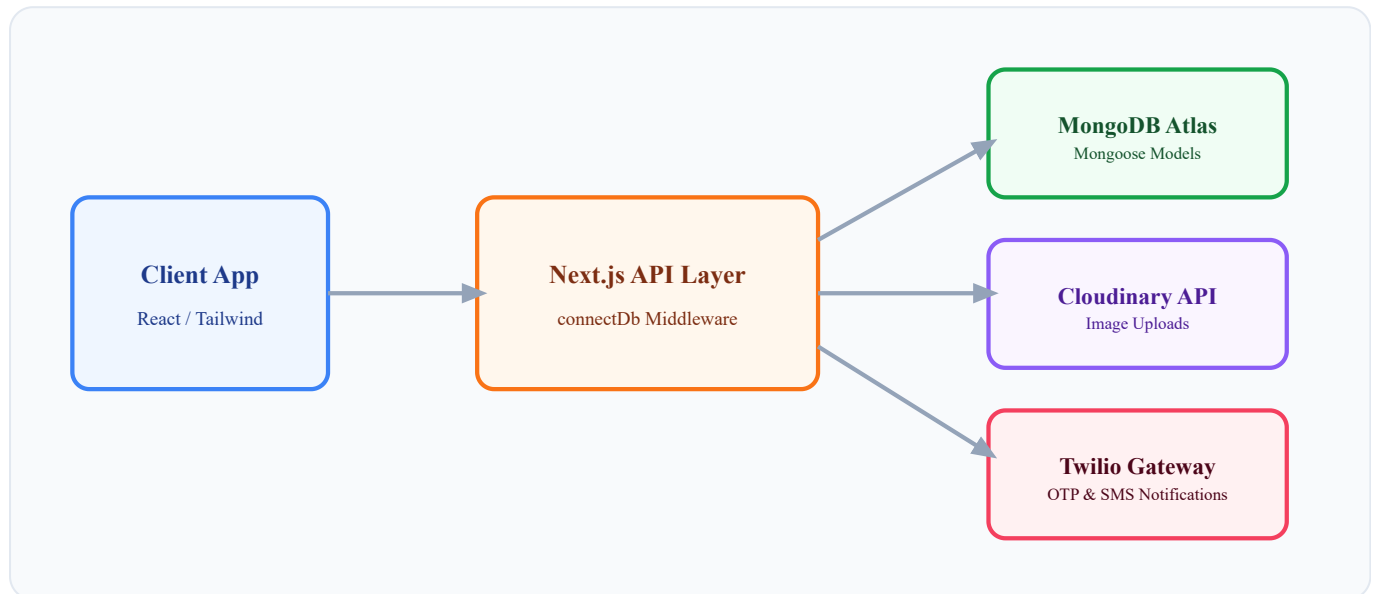
Core Technology Stack

Component	Technologies Utilized
Framework	Next.js 14.2.15 (Pages Router), React 18
Database	MongoDB Atlas with Mongoose ORM
Styling	Tailwind CSS 3.4, tw-elements, Custom HSL Gradients
Media Storage	Cloudinary API for photo upload integration
Messaging	Twilio SDK for OTP verification and automated SMS alerts
Analytics	Chart.js, react-chartjs-2, react-countup

2. Architecture & Codebase Structure

The project is structured following the standard Next.js directory guidelines, separating backend API handlers, schemas, visual layout components, styling tokens, and pages.

Architecture Flow Diagram



Project Directory Structure

```
Visitor-main/
├── components/      # Reusable React components (LetterForm, WorkerForm, charts)
├── lib/             # Third-party utilities (Cloudinary integration)
├── middleware/      # Mongoose connection middleware
├── models/          # Mongoose DB schema declarations
├── pages/           # Routing directories
│   ├── admin/       # Admin restricted dashboard and list views
│   ├── api/         # Backend API endpoint routes
│   │   ├── inward-letters/
│   │   ├── letters/
│   │   ├── workers/
│   │   └── ...       # Auth, OTP, Stats and CRUD handlers
│   ├── edit-worker/ # Dynamic route wrappers for item editing
│   ├── ...          # Other user page entry points
│   ├── _app.js       # App initialization and role gating engine
│   └── index.js      # Entry landing portal for Punit Joshi
├── styles/          # Global CSS declarations & Tailwind utilities
└── tailwind.config.js # Style custom styling extensions
```

3. Database Schema Models

The application utilizes **MongoDB** as its core database engine. Mongoose schemas shape the documents, handle validators, and automatically inject created/updated timestamps.

1. Form Data Schema (Visitor Registration)

Tracks visitor information including specific workflows depending on the purpose. File location: `models/form.js` .

Field	Type	Default	Description
photos	String	—	Cloudinary secure URL containing visitor photo.
fullName	String	—	Visitor's complete name.
phoneNo	String	—	Contact number.
purpose	String	—	Purpose category: medical, education, job, schemes, etc.
addedBy	String	—	Username of operator/user who registered the visitor.
status	String	"Pending"	Approval/follow-up state: Pending, In Progress, Completed.
followUp	String	""	Actionable logs for next administrator responses.

2. Worker Schema

Stores biographical data of workers distributed in areas/booths. File location: `models/worker.js` .

Field	Type	Required	Description
firstName / lastName	String	true	Worker's names.
primaryPhone	String	true	Unique identifier primary contact number.
position	String	true	Political/organizational role of the worker.
areaNameOrBooth	String	false	Booth registration or designated geographical area.
createdBy	String	false	Role tracker: either "admin" or "user".

3. Letter Schema (Outward Correspondence)

Tracks letters dispatched to departments or entities. File location: `models/letter.js`.

Field	Type	Required	Description
<code>subject</code>	String	true	Subject matter/purpose of the letter.
<code>inwardNumber</code>	String	true	Reference token inward reference.
<code>letterAddressedTo</code>	String	false	Target official or administrative office.
<code>assignedPerson</code>	String	false	Name of local handler managing tracking.
<code>followUpDate</code>	Date	false	Due deadline date.

4. Inward Letter Schema (Received Correspondence)

Logs all letters incoming into Punit Joshi's office. File location: `models/inward-letter.js`.

Field	Type	Required	Description
<code>inwardDate</code>	Date	true	The date the letter entered the office registry.
<code>inwardNumber</code>	String	true	Official logged serial/reference number.
<code>senderName</code>	String	false	Person/Organization who posted the document.
<code>status</code>	String	"Pending"	Enum state: Pending, In Progress, Completed.

5. Permission Schema

Provides fine-grained dashboard routing rules based on username matching. File location:

`models/permission.js`.

Field	Type	Unique	Description
<code>username</code>	String	true	Identifier key of user or administrator login.
<code>allowedPages</code>	Array [String]	false	List of routes the user is allowed to navigate (e.g. <code>"/admin/workers"</code>).

4. Backend API Reference

Backend endpoints are structured under `pages/api/`. They establish Mongoose connections upon invocation via the `connectDb` middleware.

Authentication & System APIs

Endpoint	Method	Behavior / Payload
<code>/api/login</code>	POST	Checks hardcoded parameters or MongoDB user. Returns role, allowed routes, status messages.
<code>/api/stats</code>	GET	Aggregates metrics for the Admin Dashboard: total counts, today's visitors, purpose percentages.
<code>/api/send-sms</code>	POST	Sends SMS messages via Twilio integration using <code>to</code> and <code>message</code> fields.

Resource CRUD APIs

Endpoint	Method	Behavior / Payload
<code>/api/visitors</code>	GET	Lists registered visitors. Supports filtering by <code>addedBy</code> , search query, or pagination.
<code>/api/workers</code>	GET	Retrieves worker profiles. Can filter by creator role (admin/user) to protect data access.
<code>/api/workers/add</code>	POST	Validates input and saves a new worker document in the MongoDB collection.
<code>/api/letters/add</code>	POST	Creates outward letter reference trackers in the system registry database.
<code>/api/inward-letters/add</code>	POST	Stores a new inward letter with scanned photo URL and status tracking parameters.

Access Permissions Handler

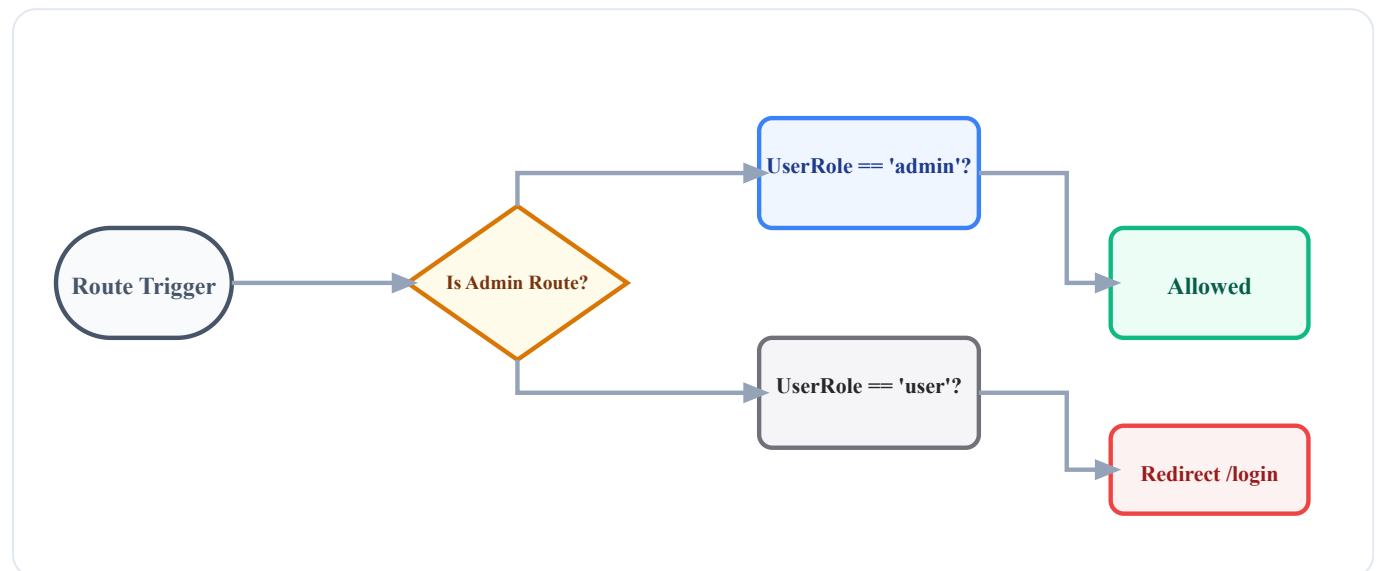
Provides tools to retrieve and update routes allowed for specific sub-operators.

```
// Example response from GET /api/permissions/user?username=MKulkarni
{
  "success": true,
  "username": "MKulkarni",
  "allowedPages": [
    "/admin",
    "/admin/visitorTable",
    "/admin/workers",
    "/admin/addWorker"
  ]
}
```


5. Authorization & Gating Flows

The application enforces strict page-level routing rules in `pages/_app.js`. Whenever a path transition is triggered, the system reads authentication metadata from `localStorage`.

Access Control Flowchart



Operator & User Route Syncing

At startup, if a user logged in with the role `"admin"`, their permission records are asynchronously pulled from the backend using the current username. This ensures that even if permissions are modified in the panel, they apply immediately without requiring a full logout.

```
// Dynamic sync snippet on application mount
useEffect(() => {
  if (role === "admin" && username) {
    fetch('/api/permissions/user?username=' + encodeURIComponent(username))
      .then((res) => res.json())
      .then((data) => {
        if (data.success && data.allowedPages) {
          localStorage.setItem("allowedPages", JSON.stringify(data.allowedPages));
        }
      });
  }
}, []);
```

6. Configuration & Developer Guidelines

To set up the project locally or deploy it to a production server, several environment parameters must be configured.

Required Environment Parameters (.env.local)

Parameter	Purpose / Value Description
MONGO_URI	MongoDB database connection URI string (MDB Atlas connection string).
ADMIN_USERNAME	Super administrator username identifier.
ADMIN_PASSWORD	Super administrator password.
USER_USERNAME	Default general user username.
USER_PASSWORD	Default general user password.
CLOUDINARY_CLOUD_NAME	Cloudinary account identifier for media storage.
CLOUDINARY_API_KEY	Cloudinary API authentication credentials.
CLOUDINARY_API_SECRET	Cloudinary API Secret key.
TWILIO_ACCOUNT_SID	Twilio account unique string.
TWILIO_AUTH_TOKEN	Twilio SDK authorization key.
TWILIO_PHONE_NUMBER	Twilio virtual outbound sender telephone number.

Developer Environment Command Scripts

Manage application builds and dev servers using these commands at the root of the project:

- **Start Local Development Server:** `npm run dev`
- **Build Production Application:** `npm run build`
- **Launch Production Server:** `npm start`
- **Static Lint Audit:** `npm run lint`